

[← Back to calendar](#)

9% progress

 **Mandatory deliverable**

LAB | State Management & Collaboration

Lab

Skills you will learn

Infrastructure as Code: Terraform

Development Workflow: Problem Solving

Lab M4.04 - State Management & Operations

Repository: <https://github.com/cloud-engineering-bootcamp/ce-lab-terraform-state-ops>**Activity Type:** Individual **Estimated Time:** 45-60 minutes **Submission:** GitHub Repository

Learning Objectives

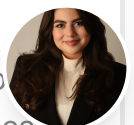
- ☐ Use terraform state commands effectively
- ☐ Handle state drift and reconciliation
- ☐ Import existing AWS resources into state
- ☐ Move resources between state files
- ☐ Remove resources from state without destroying
- ☐ Troubleshoot common state issues

Prerequisites

- ☐ Completed Lab M4.03 (Remote State)
- ☐ Understanding of Terraform state concepts
- ☐ Existing AWS resources to import

Introduction

State management is critical for Terraform operations. Resources drift from configuration, external changes happen, and you need to import existing infrastructure. This lab teaches



essential state management skills.

Your Task

What you'll practice:

- Inspecting state with `terraform state list` and `show`
- Handling state drift with `terraform refresh`
- Importing existing resources
- Moving resources between state files
- Removing resources from state
- Recovering from state issues

Time limit: 45-60 minutes

Step-by-Step Instructions

Step 1: Setup Lab Environment

```
mkdir -p ~/ce-labs/m4-04-state-ops
cd ~/ce-labs/m4-04-state-ops

# Create basic configuration
cat > main.tf <<'EOF'
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "managed" {
  bucket = "state-ops-managed-bucket"

  tags = {
    Name      = "Managed Bucket"
    ManagedBy = "Terraform"
  }
}
```



```
resource "aws_s3_bucket" "example1" {
  bucket = "state-ops-example1"

  tags = {
    Name = "Example 1"
  }
}

resource "aws_s3_bucket" "example2" {
  bucket = "state-ops-example2"

  tags = {
    Name = "Example 2"
  }
}
EOF

terraform init
terraform apply -auto-approve
```

Step 2: Explore State Commands

List all resources in state
`terraform state list`

[Copy](#)[Explain](#)

Expected output:
aws_s3_bucket.example1
aws_s3_bucket.example2
aws_s3_bucket.managed

Show specific resource details
`terraform state show aws_s3_bucket.managed`

Show all state in JSON
`terraform show -json | jq '!`

Step 3: Create Resource Outside Terraform

Create bucket manually via AWS CLI
`aws s3api create-bucket \`
`--bucket state-ops-unmanaged \`
`--region us-east-1`

Tag it
`aws s3api put-bucket-tagging \`
`--bucket state-ops-unmanaged \`
`--tagging 'TagSet=[{Key=Name,Value=Unmanaged Bucket}]'`



```
# Verify it exists
aws s3 ls | grep state-ops-unmanaged
```

Step 4: Import Existing Resource

Add to `main.tf` :

```
resource "aws_s3_bucket" "imported" {
  bucket = "state-ops-unmanaged"

  tags = {
    Name      = "Imported Bucket"
    ManagedBy = "Terraform"
  }
}
```

[Copy](#)[Explain](#)

Import into state:

```
# Import the resource
terraform import aws_s3_bucket.imported state-ops-unmanaged

# Verify import
terraform state list | grep imported

# Check if configuration matches
terraform plan
```

[Copy](#)[Explain](#)

Expected: Plan may show tag updates (imported bucket vs config).

Step 5: Handle State Drift

```
# Manually modify resource outside Terraform
aws s3api put-bucket-tagging \
  --bucket state-ops-example1 \
  --tagging 'TagSet=[{Key=Name,Value=Modified Outside},
{Key=Manual,Value=Change}]'

# Detect drift
terraform plan

# Should show:
# ~ update in-place
# ~ tags = {
#   + Manual = "Change"
#   ~ Name   = "Example 1" -> "Modified Outside"
```



```
# }

# Refresh state to match reality
terraform refresh

# Re-plan (brings back to desired state)
terraform plan

# Apply to fix drift
terraform apply -auto-approve
```

Step 6: Move Resources in State

```
# Rename resource in configuration
# Change aws_s3_bucket.example1 to aws_s3_bucket.primary

# Move in state (prevents destroy/recreate)
terraform state mv \
  aws_s3_bucket.example1 \
  aws_s3_bucket.primary

# Update configuration to match
sed -i 's/example1/primary/g' main.tf

# Verify no changes needed
terraform plan

# Should show: No changes
```

 Copy Explain

Step 7: Remove Resource from State

```
# Remove resource from state WITHOUT destroying it
terraform state rm aws_s3_bucket.example2

# Verify removed from state
terraform state list | grep example2
# Should return nothing

# Check AWS - bucket still exists
aws s3 ls | grep state-ops-example2
# Still there!

# Terraform now doesn't manage it
terraform plan

# No mention of example2
```

 Copy Explain

Step 8: Pull and Push State (Advanced)



Pull current state

```
terraform state pull > state-backup.json
```

[Copy](#)[Explain](#)

View state

```
cat state-backup.json | jq '.resources'
```

Make manual modification (DANGEROUS - example only)

Edit state-backup.json

Change a tag value

Push modified state (USE WITH EXTREME CAUTION)

```
terraform state push state-backup.json
```

⚠ **Warning:** Never manually edit state in production!

Step 9: Replace Resource

Force recreation of resource

```
terraform apply -replace="aws_s3_bucket.primary"
```

[Copy](#)[Explain](#)

Terraform will:

1. Destroy old bucket

2. Create new bucket with same configuration

Step 10: Handle Locked State

Simulate locked state by starting apply in background

```
terraform apply &  
APPLY_PID=$!
```

[Copy](#)[Explain](#)

Wait a moment

```
sleep 2
```

Try another operation (will fail)

```
terraform plan
```

Error: Error acquiring the state lock

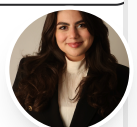
Get lock info

```
terraform force-unlock <LOCK_ID>
```

Clean up background process

```
kill $APPLY_PID
```

Common State Operations Reference



List resources

terraform state list

[Copy](#)[Explain](#)

Show resource details

terraform state show <resource_address>

Move resource

terraform state mv <source> <destination>

Remove resource

terraform state rm <resource_address>

Import resource

terraform import <resource_address> <resource_id>

Replace resource

terraform apply -replace="<resource_address>"

Refresh state

terraform refresh

Pull state

terraform state pull > state.json

Push state (dangerous!)

terraform state push state.json

Force unlock

terraform force-unlock <lock_id>

Troubleshooting

Issue 1: Resource Not Found After Import

Problem: Imported resource, but plan wants to create it.

Solution:

Ensure resource address matches exactly

terraform state list

Check: aws_s3_bucket.imported

Configuration must match this name exactly

[Copy](#)[Explain](#)

Issue 2: State Drift After Manual Changes

Problem: Resources differ from configuration.



Solution:

```
# Option 1: Update configuration to match reality
# Edit main.tf

# Option 2: Apply configuration to fix drift
terraform apply
```

[Copy](#)[Explain](#)

Issue 3: Cannot Remove Resource

Error:

```
Error: Error removing resource: resource has dependencies
```

Solution:

```
# Remove dependents first
terraform state rm <dependent_resource>
terraform state rm <main_resource>
```

[Copy](#)[Explain](#)

Documentation & Submission

Create README.md

```
# Lab M4.04 - State Management Operations

## Operations Practiced

### 1. State Inspection
- `terraform state list` - List all resources
- `terraform state show` - View resource details

### 2. Resource Import
- Imported existing S3 bucket into state
- Verified configuration matches

### 3. State Drift Handling
- Detected manual changes outside Terraform
- Used `terraform refresh` and `apply` to reconcile

### 4. Resource Movement
- Renamed resource in state without recreation
- Used `terraform state mv`
```



5. Resource Removal

- Removed resource from state without destroying
- Used `terraform state rm`

Key Learnings

- State is source of truth for Terraform
- Manual changes cause drift
- Import brings existing resources under management
- State operations require care and backups

Safety Tips

- Always backup state before manual operations
- Use remote state with versioning
- Test state operations in non-production first
- Never manually edit state files

Create Commands Cheat Sheet

Create `state-commands-cheatsheet.md` :

Terraform State Commands Cheat Sheet

 Copy

 Explain

Inspection

```
\\`bash
terraform state list          # List all resources
terraform state show <resource> # Show resource details
terraform show                # Show entire state
terraform show -json | jq     # State as JSON
\\`
```

Modification

```
\\`bash
terraform state mv <src> <dst> # Rename resource
terraform state rm <resource>   # Remove from state
terraform state pull > backup.json # Backup state
\\`
```

Import & Replace

```
\\`bash
terraform import <resource> <id> # Import existing
terraform apply -replace="<resource>" # Force recreate
\\`
```

Troubleshooting

```
\\`bash
terraform refresh          # Sync state with reality
terraform force-unlock <lock_id> # Force unlock
\\`
```



Bonus Challenges

Challenge 1: Batch Operations

Write script to import multiple resources at once.

Challenge 2: State Migration

Move resources between two separate state files.

Challenge 3: State Recovery

Practice recovering from corrupted state using versions.

Grading Rubric (100 points)

Criteria	Points
State inspection commands	15
Import existing resource	20
Handle state drift	20
Move resources in state	20
Remove resources safely	15
Documentation	10

Key Takeaways

✅ **State list/show** - Inspect current state ✅ **Import** - Bring existing resources under management ✅ **Refresh** - Detect and handle drift ✅ **Move** - Rename without recreating ✅ **Remove** - Stop managing without destroying ✅ **Backup** - Always backup before manual operations

Next Lab: M4.05 - Create Your First Terraform Module

LAB | State Management & Collaboration

Lab

URL

Enter the URL here



Submit lab

Previous

Week 4 - Infrastructure as Code with Terraform: Day 2 - State Management & Remote Backends
State Management & Remote State Quiz

